

OpenPLC Editor

用户手册

简体中文译本

原文：LPC Manager User Manual Version 4
(基于 Beremiz / IEC 61131-3)

原作者：SMARTEH d.o.o.
Copyright © 2010-2016, SMARTEH d.o.o.
原文文档版本：4 (2016 年 1 月)
中文译本收录于 OpenPLC Editor 仓库

目录

1 关于本文档

1.1 谁应阅读本文档

1.2 术语

1.2.1 基于 LPC 系列产品的术语

1.2.2 基于 LPC Manager 的术语

1.2.3 本文档所用约定

2 LPC MANAGER 软件

2.1 简介

2.2 LPC Manager 编辑器

2.3 主菜单

2.3.1 File (文件)

2.3.2 Edit (编辑)

2.3.3 Display (显示)

2.3.4 Help (帮助)

2.4 工具栏

2.5 Project (项目) 窗口

2.5.1 Topology (拓扑)

2.5.2 Project (程序结构窗口)

2.5.3 Project (实例窗口)

2.5.4 Editor workspace (编辑器工作区)

2.6 库 (Library)

2.6.1 标准功能块

2.6.2 附加功能块

2.6.3 类型转换

2.6.4 数值运算

2.6.5 算术运算

2.6.6 时间运算

2.6.7 位移运算

2.6.8 按位运算

2.6.9 选择

2.6.10 比较

2.6.11 字符串

2.6.12 本地 POU

2.6.13 LPC POU

2.6.14 用户自定义 POU

2.7 Debugger (调试器)

2.8 Search (搜索)

2.9 Console (控制台)

2.10 PLC Log (PLC 日志)

3 编程语言

3.1 IL - Instruction List (指令表)

3.2 ST - Structured Text (结构化文本)

3.3 FBD - Function Block Diagram (功能块图)

3.4 LD - Ladder Diagram (梯形图)

3.5 SFC - Sequential Function Chart (顺序功能图)

附录 A – 错误报告

附录 B – 文档历史

译者说明

1 关于本文档

LPC Manager 用户手册说明如何使用应用程序 LPC Manager。

1.1 谁应阅读本文档

如果您是 LPC Manager 软件的新用户，希望入门使用，或正从先前版本升级，则应阅读本文档。为正确理解本文档，至少需要了解 LPC 硬件的基本知识。为此，强烈建议观看 LPC Getting Started 视频（随 LPC Smarteh IDE 安装包一并安装），或参加由经认证的 SMARTEH d.o.o. 培训师提供的正式培训。在该短期课程中，您将学习全部基础内容，并有机会成为经认证的 SMARTEH d.o.o. 集成商。您不仅能抢先掌握要点，还将加入不断壮大的 LPC 用户群体，有机会直接向专家提问，并与世界各地的其他集成商及公司建立联系…… 注意：由于 LPC Manager 基于 IEC 61131-3 国际标准，更详细的信息请参阅 PLC（可编程逻辑控制器）编程语言国际标准 IEC 61131-3。如需了解更多信息，请致电 +386 5 388 44 00，或发送电子邮件至 info@smarteh.si，预约 LPC 认证培训名额。

1.2 术语

本手册通篇使用多种表述。以下为其中部分术语的说明。

1.2.1 基于 LPC 系列产品的术语

LONGO™ — 产品系列（硬件与软件）名称，为 SMARTEH d.o.o. 的商标。LPC™ — Longo Programming Controller（Longo 可编程控制器），为 SMARTEH d.o.o. 的商标。IDE — 集成开发环境（Integrated Development Environment）。LPC-2 Programming Controller — 硬件模块系列（MCU 模块、I/O 模块及其他模块）。LPC Smarteh IDE、LPC Manager — 属于 LPC 系列软件成员。

1.2.2 基于 LPC Manager 的术语

Beremiz — 面向自动化的自由软件框架（<http://www.beremiz.org>）IEC 61131-3 — 可编程控制器及编程语言的国际标准 PLC — Programmable Logic Controller（可编程逻辑控制器）MCU — Main Control Unit（主控单元）POU — Program Organization Unit（程序组织单元）编程语言：IL — Instruction List（指令表）ST — Structured Text（结构化文本）LD — Ladder Diagram（梯形图）FBD — Function Block Diagram（功能块图）SFC — Sequential Function Chart（顺序功能图）True — 逻辑 1，接通，有效，高电平状态 False — 逻辑 0，断开，无效，低电平状态

1.2.3 本文档所用约定

本文档中出现的部分内容会以特殊外观呈现，以便与正文区分。具体如下：斜体 — 用于标注重要关键词。NEW: — 用于标注相对先前版本变更最多的章节。

2 LPC MANAGER 软件

2.1 简介

LPC Manager 软件 (LONGO Programmable Controller Manager software) 用于对 Smarteh 控制器的 LPC-2 系列进行编程。在 LPC Smarteh IDE 中创建配置后, 可通过以下方式启动 LPC Manager:

- 单击 Program (程序) 按钮, 或
- 选择某个项目配置, 然后单击 Program in LPC Manager (在 LPC Manager 中编程) 命令行。

本软件基于 Beremiz 开源软件, 并针对 Smarteh LPC 控制器进行适配, 支持 IEC 61131-3 标准编程语言: IL (指令表)、ST (结构化文本)、LD (梯形图)、FBD (功能块图) 和 SFC (顺序功能图)。LPC Manager 软件易于使用, 并在 LPC 控制器应用软件的编程、调试、监视与趋势分析方面提供多种功能。

2.2 LPC Manager 编辑器

LPC Manager 软件由以下部分组成:

- 主菜单: File (文件)、Edit (编辑)、Display (显示)、Help (帮助)
- 工具栏: Save (保存)、Print (打印)、Undo (撤销)、Redo (重做)、Cut (剪切)、Copy (复制)、Paste (粘贴)、Search in Project (在项目中搜索); Simulate (仿真)、Debug (调试); Select an object (选择对象)、Move the view (移动视图)、Create a new comment (创建新注释)、Create a new variable (创建新变量)、Create a new block (创建新功能块)、Create a new connection (创建新连接)
- Topology (拓扑) 与 Project (项目) 窗口
- Variables (变量) 与 Editor workspace (编辑器工作区) 窗口
- Search (搜索)、Console (控制台) 与 PLC Log (PLC 日志) 窗口
- Library (库) 与 Debugging (调试) 窗口

2.3 主菜单

在主菜单中可找到 File (文件)、Edit (编辑)、Display (显示) 和 Help (帮助) 选项。

2.3.1 File (文件)

Save (保存) - 保存当前打开的工作区。 Close Tab (关闭标签页) - 关闭当前打开的工作区。 Page Setup (页面设置) - 设置打印页面。 Preview (预览) - 工作区的打印预览。 Print (打印) - 打印当前打开的工作区。 Quit (退出) - 退出 LPC Manager。

2.3.2 Edit (编辑)

Undo (撤销) - 撤销工作区中的上一次更改。 Redo (重做) - 恢复撤销操作的效果。 Cut (剪切) - 剪切工作区中选定的元素。 Copy (复制) - 复制工作区中选定的元素。 Paste (粘贴) - 将 (先前复制的) 元素粘贴到工作区中。 Find (查找) - 搜索元素 Find Next (查找下一个) - 搜索下一个元素 Find Previous (查找上一个) - 搜索上一个元素 Search in Project (在项目中搜索) - 在项目中搜索元素。 Add Element (添加元素) - 将元素 (Data Type、Function、Function Block、Program、Configuration) 添加到 Types (类型) 窗口下的相应项中。 Select All (全选) - 选择工作区中的全部元素。 Delete (删除) - 从工作区中删除元素。

2.3.3 Display (显示)

Refresh (刷新) - 刷新工作区。 Clear Errors (清除错误) - 清除程序错误 Zoom (缩放) - 窗口缩放设置 (12 .. 800%)。 Reset Perspective (重置透视图) - 将程序窗口重置为默认布局

2.3.4 Help (帮助)

About (关于) - Beremiz 的主要信息。

2.4 工具栏

Toolbar1 (工具栏 1) - 标准 Windows 图标 (Save、Print、Undo、Redo、Cut、Copy、Paste、Search in Project)。 Toolbar2 (工具栏 2) - 与 PLC 相关的执行功能 (可用元素集取决于 USB 连接或仿真状态)。可在 Off-line (离线) 模式、Simulation (仿真) 模式、Application (应用) 模式、Bootloader (引导加载程序) 模式下使用。 Toolbar3 (工具栏 3) - 编辑器工作区元素的主要图形图标 (可用元素集取决于所选编程语言)。

Simulate PLC (仿真 PLC): 按下 “Simulate” 按钮, 启动在 PC 上运行的 PLC 仿真。所有调试功能均受支持, 与连接 LPC-2 控制器时的在线调试模式相同。 Build project into build folder (将项目构建到 build 文件夹): 按下 “Build” 按钮开始构建项目。“Log Console” 会显示各个构建步骤。构建结果为可执行代码, 名称与项目名称相同, 位于项目的 build 目录中。 Transfer PLC (传送 PLC): 本命令及后续命令仅在 PC 通过 USB 编程电缆连接到 LPC-2 控制器, 且表示 USB 连通性的蓝色 LED 点亮时可用。按下 “Transfer” 按钮, 将可执行应用程序代码传送到 LPC-2 控制器。 Start PLC (启动 PLC): 当已连接 LPC-2 控制器上的操作模式开关处于 “RUN” 位置时, 本命令可用。按下 “Run” 按钮后, 控制器将开始执行应用程序。绿色 “RUN” LED 将点亮。 Stop running PLC (停止正在运行的 PLC): 按下 “Stop” 按钮后, 已连接的 LPC-2 控制器将停止全部 LPC-2 控制器进程。绿色 “RUN” LED 以及已连接模块的输出将进入关断状态。

Toolbar3 (工具栏 3) 详情: Select an object (选择对象) - 在 POU 内选择一个或多个对象的标准工具。 Move the view (移动视图) - 将 POU 内的当前视图移动到所需方向。适用于 LD、FBD 和 SFC。 Create a new comment (创建新注释) - 在 POU 中插入新注释。适用于 LD、FBD 和 SFC。 Create a new power rail (创建新电源轨) - 在 POU 中插入电源轨 (左侧或右侧)。适用于 LD 和 SFC。 Create a new coil (创建新线圈) - 在 POU 中插入线圈。适用于 LD。 Create a new contact (创建新触点) - 在 POU 中插入触点。适用于 LD 和 SFC。 Create a new variable (创建新变量) - 在 POU 中插入变量。适用于 LD 和 SFC。 Create a new block (创建新功能块) - 在 POU 中插入功能块。适用于 LD、FBD 和 SFC。 Create a new connection (创建新连接) - 在 POU 中插入连接。适用于 LD、FBD 和 SFC。 Create a new initial step (创建新初始步) - 在 POU 中插入初始步。适用于 SFC。 Create a new step (创建新步) - 在 POU 中插入新步。适用于 SFC。 Create a new transition (创建新转换) - 在 POU 中插入转换。适用于 SFC。 Create a new action block (创建新动作块) - 在 POU 中插入动作块。适用于 SFC。 Create a new divergence (创建新分支) - 在 POU 中插入分支。适用于 SFC。 Create a new jump (创建新跳转) - 在 POU 中插入跳转。适用于 SFC。

2.5 Project (项目) 窗口

Project window (项目窗口) - 由下文所述 Topology (拓扑) 与 Project (项目) 组成的一组窗口。

2.5.1 Topology (拓扑)

Topology (拓扑) - 表示当前控制器配置的全部变量, 该配置由 LPC Smarteh IDE 软件组成。 Project name (MC8) (项目名称) - 在 LPC Smarteh IDE 配置中定义的项目名称 CanOpen - CANopen

变量与通信设置。 Right (右侧) - 输入/输出模块变量。 Left (左侧) - 网络模块变量。 Devices (设备) - 智能外设模块变量 On Board (板载) - MCU 板载变量。

Build (构建) - Transfer (传送) 流程: 首次打开 LPC Manager 时 (针对对应的 LPC Smarteh IDE 配置), Toolbar 2 中提供两个按钮: Simulate 与 Build。在 POU 内创建所需应用程序后, 必须对该应用程序进行构建。此过程可在 Console 窗口中观察。若代码无错误, 则会在项目文件夹中生成相应文件。下一步是将生成的二进制代码传送到 LPC-2 控制器; 控制器须通过 USB 编程电缆 (USB A 型公头转 USB B 型公头) 连接到 USB 端口。此时 USB 连通性 (蓝色) LED 点亮, Toolbar 2 中会增加 Transfer 按钮, 并且 Simulate 变为 Stop。单击 Transfer 按钮, 并在 LOG console 窗口中观察进度。若一切正常, 将报告消息 “PLC transferred successfully”, 并且控制器会自动启动 (绿色 RUN LED 点亮)。如果向 MCU 传送程序时遇到问题, 请将 MCU 切换到 STOP 位置 (控制器将进入 bootloader 模式, 绿色 “RUN” LED 应熄灭), 然后再次按下 Transfer。重要: 传送完成后, 全部内部存储器 (retain 数据、RTC 等) 将被擦除。

2.5.2 Project (程序结构窗口)

Project (MC8) - 主项目属性与说明 (Project、Author、Graphics、Miscellaneous) Data Types (数据类型) - 用户定义数据类型 (Directly、Subrange、Enumerated、Array、Structure)。 Functions (功能) - 用户定义 POU 功能 (IL、ST、LD、FBD) Function Blocks (功能块) - 用户定义 POU 功能块 (IL、ST、LD、FBD、SFC)。 Programs (程序) - POU 程序 (IL、ST、LD、FBD、SFC)。 Resources (资源) - 包含变量列表 (全局变量)、任务与实例 (用于执行项目中的 POU 程序)。

添加新建项 (DataTypes、Functions、Function Blocks、Programs 和 Resources): New POU (新建 POU) - 可通过单击右下角的大加号添加新 POU。必须定义 POU 类型与编程语言。POU 名称可以更改。

右键菜单 (Project name): Paste POU (粘贴 POU) - 可将已复制的 POU (例如 Function block) 粘贴到对应的 Types 分区中。也可从文本文件导入 POU (先前导出的 POU)。

右键菜单 (DataTypes、Functions、Function Blocks、Programs 和 Resources): Add DataType (添加数据类型) - 可创建新的 DataType。 Add POU (添加 POU) - 可创建新的 POU。必须定义编程语言。POU 名称与 POU 类型可以更改。 Paste POU (粘贴 POU) - 可将已复制的 POU (例如 Function block) 粘贴到对应的 Types 分区中。也可从文本文件导入 POU (先前导出的 POU)。 Add Resources (添加资源) - 可创建新的 Resources。

右键菜单 (Function、Function block 和 Program): 右键菜单 (Function block) 的内容取决于编程语言与 POU 类型。 Add transition (添加转换) - (仅适用于 SFC 语言) Add action (添加动作) - (仅适用于 SFC 语言) Copy POU (复制 POU) - 可将 Function block 复制 (粘贴) 到 Function Blocks 分区中。 Change POU Type To (将 POU 类型更改为) - 可将所选 Function 的 POU 类型更改为 Function block 或 Program, 也可将 Function block 更改为 Program。 Rename (重命名) - 重命名所选 Function block。 Delete (删除) - 删除所选 Function block。 提示: 可将已复制 Function block 的内容 (剪贴板) 保存为文本文件, 以便备份或在其他 LPC Manager 应用程序中使用。

双击 (Project name、DataTypes、Functions、Function Blocks、Programs 和 Resources):

- 双击任意数据类型或 POU, 会在编辑器工作区中打开所选数据类型或 POU。
- 双击项目名称, 会在编辑器工作区中打开 Config variables (配置变量) 与 Project properties (项目属性)。

2.5.3 Project (实例窗口)

Instances (实例) - 包含已构建并传送到目标 LPC-2 控制器的应用程序映像。用于在线图形调试模式下的图形化呈现。全部变量与内部逻辑连接均可在编辑工作区中以逻辑结构形式进行动画显示, 也可在调试窗口列表

中以数值形式显示，或在实时图形趋势窗口中显示。可通过单击眼镜图标启动调试。可将变量强制为所需数值。

2.5.4 Editor workspace (编辑器工作区)

Editor workspace (编辑器工作区) - 用于对项目窗口中的全部元素 (POU、数据类型、配置、资源、拓扑及其他变量、实例等) 进行编辑、设置、编程与调试。编辑元素会在独立窗口中打开，与工作区中列出的那些元素分开显示。

快捷键: CTRL+Scroll Up - 放大 CTRL+Scroll Down - 缩小 CTRL+Down Arrow - 向下滚动 CTRL+Up Arrow - 向上滚动 CTRL+Right Arrow - 向右滚动 CTRL+Left Arrow - 向左滚动
双击元素之间的连接线可优化连接线长度 (最短线) CTRL+C、CTRL+V - 复制 - 粘贴元素 (在按下 CTRL+V 之前先单击选定位置; 光标形状为十字)

警告! 在编辑字段中使用非标准字符 (例如 č, š, ž 等) 可能导致编辑器运行异常。

提示: 将光标置于目标功能块上时, 该功能块的名称及对应变量会显示在光标旁的小弹出窗口中。

VARIABLES (变量) - Variables 窗口包含相关 POU

中使用的所有变量, 用于通过其基本数据类型声明标识数据对象。快捷键: 双击 Type (类型) 字段可打开弹出栏, 以选择变量的 Base type (基本类型) 或 User Data type (用户数据类型)。

2.6 库 (Library)

库 (Library) —— 库包含多组标准函数与用户自定义函数。它们支持在 POU 内以不同的可编程控制器编程语言使用。

块属性 (BLOCK PROPERTIES)
可通过双击功能块打开“块属性”弹出窗口。部分功能块的输入数量可多于默认值, 可在 Inputs 字段中选择 (例如 ADD 块)。此外, 功能块的执行顺序 (Execution Order) 也可由程序员自定义。所有功能块都带有额外的 Execution Control 复选框。若勾选该选项, 将新增两个引脚 (EN——输入, ENO——输出), 用于动态控制其执行。

2.6.1 标准功能块

SR - SR 双稳态——SR 双稳态为置位优先 (Set dominates) 的锁存器。

```
( BOOL:S1, BOOL:R ) => ( BOOL:Q1 )
```

该函数表示标准的置位优先置位/复位触发器。当输入 S1 为 TRUE 且输入 R 为 FALSE 时, 输出 Q1 变为 TRUE。同样, 当输入 S1 为 FALSE 且输入 R 为 TRUE 时, 输出 Q1 变为 FALSE。在上述任一转换之后, 当 S1 与 R 均回到 FALSE 时, 输出 Q1 将保持先前状态, 直至出现新的条件。若对两个信号同时施加 TRUE, 则输出 Q1 被强制为 TRUE (置位优先)。

RS - RS 双稳态——RS 双稳态为复位优先 (Reset dominates) 的锁存器。

```
( BOOL:S, BOOL:R1 ) => ( BOOL:Q1 )
```

该函数表示标准的复位优先置位/复位触发器。当输入 S 为 TRUE 且输入 R1 为 FALSE 时, 输出 Q1 变为 TRUE。同样, 当输入 S 为 FALSE 且输入 R1 为 TRUE 时, 输出 Q1 变为 FALSE。在上述任一转换之后, 当 S 与 R1 均回到 FALSE 时, 输出 Q1 将保持先前状态, 直至出现新的条件。若对两个信号同时施加 TRUE, 则输出 Q1 被强制为 FALSE (复位优先)。

SEMA - 信号量——信号量提供一种机制, 使软件元素能够对特定资源进行互斥访问。

```
( BOOL:CLAIM, BOOL:RELEASE ) => ( BOOL:BUSY )
```


该功能块实现信号量功能。通常用于事件同步。BUSY 输出由 CLAIM 输入上的 TRUE 条件激活，并由 RELEASE 输入上的 TRUE 条件去激活。

R TRIG - 上升沿检测器——当检测到上升沿时，输出产生一个单脉冲。

```
( BOOL:CLK ) => ( BOOL:Q )
```

该函数为上升沿检测器。当在 CLK 输入上检测到 0 到 1 (或 FALSE 到 TRUE，或 OFF 到 ON) 的条件时，输出 Q 变为 TRUE，并在一个完整扫描周期内保持该状态。

F TRIG - 下降沿检测器——当检测到下降沿时，输出 Q 产生一个单脉冲。

```
( BOOL:CLK ) => ( BOOL:Q )
```

该函数为下降沿检测器。当在 CLK 输入上检测到 1 到 0 (或 TRUE 到 FALSE，或 ON 到 OFF) 的条件时，输出 Q 变为 TRUE，并在一个完整扫描周期内保持该状态。

CTU / CTU_DINT, CTU_LINT, CTU_UDINT, CTU_ULINT - 加计数器——加计数器可用于在计数值达到最大值时发出信号。

```
CTU: ( BOOL:CU, BOOL:R, INT:PV ) => ( BOOL:Q, INT:CV )
```

```
CTU_DINT: ( BOOL:CU, BOOL:R, DINT:PV ) => ( BOOL:Q, DINT:CV )
```

```
CTU_LINT: ( BOOL:CU, BOOL:R, LINT:PV ) => ( BOOL:Q, LINT:CV )
```

```
CTU_UDINT: ( BOOL:CU, BOOL:R, UDINT:PV ) => ( BOOL:Q, UDINT:CV )
```

```
CTU_ULINT: ( BOOL:CU, BOOL:R, ULINT:PV ) => ( BOOL:Q, ULINT:CV )
```

CTU 函数表示加计数器。CU 输入上的上升沿会使计数器加一。当施加于输入 PV 的设定值到达时，输出 Q 变为 TRUE。在 R 输入上施加 TRUE 信号会将计数器复位为零 (异步复位)。CV 输出报告当前计数值。

CTD / CTD_DINT, CTD_LINT, CTD_UDINT, CTD_ULINT - 减计数器

```
CTD: ( BOOL:CD, BOOL:LD, INT:PV ) => ( BOOL:Q, INT:CV )
```

(及其变体) CTD 函数表示减计数器。CD 输入上的上升沿会使计数器减一。当当前计数值小于或等于零时，输出 Q 变为 TRUE。在 LD (LOAD) 输入上施加 TRUE 信号会将计数器装载为输入 PV 上的值 (异步装载)。CV 输出报告当前计数值。

CTUD / 变体 - 加减计数器

```
CTUD: ( BOOL:CU, BOOL:CD, BOOL:R, BOOL:LD, INT:PV ) => ( BOOL:QU, BOOL:QD, INT:CV )
```

该函数表示可编程加减计数器。CU (COUNT-UP) 输入上的上升沿使计数器加一，而 CD (COUNT-DOWN) 输入上的上升沿使当前值减一。在 R 输入上施加 TRUE 信号会将计数器复位为零。LD 信号为 TRUE 时，会将计数器装载为施加于输入 PV (PROGRAMMED VALUE) 的值。当当前计数值大于或等于设定值时，QU 输出变为有效。当当前值小于或等于零时，QD 输出变为有效。CV 输出报告当前计数值。

TP - 脉冲定时器——脉冲定时器可用于生成给定持续时间的输出脉冲。

```
( BOOL:IN, TIME:PT ) => ( BOOL:Q, TIME:ET )
```

当在 IN 输入上检测到上升沿转换时，输出 Q 立即变为 TRUE。该状态持续至设定时间 PT 届满。PT 届满后，若输入 IN 仍为有效，则输出 Q 保持 ON 状态，否则输出 Q 返回 OFF 状态。该定时器不可重触发。ET 输出报告当前已经过的时间。

TON - 接通延时定时器

```
( BOOL:IN, TIME:PT ) => ( BOOL:Q, TIME:ET )
```

将该函数的输入信号 IN 置为有效即可启动定时器。当设定时间 PT 届满且输入 IN 仍为有效时，输出 Q 变为 TRUE。该状态将持续至输入 IN 被释放。若在时间届满前释放 IN 输入，定时器将被清零。ET 输出报告当前已经过的时间。

TOF - 断开延时定时器

```
( BOOL:IN, TIME:PT ) => ( BOOL:Q, TIME:ET )
```

将输入信号 IN 置为有效会立即激活输出 Q。释放输入 IN 将开始计时。当设定时间 PT 届满且输入 IN 仍为释放状态时，输出 Q 变为 FALSE。若在时间届满前再次将 IN 输入置为有效，定时器将被清零，且输出 Q 保持为 TRUE。ET 输出报告当前已经过的时间。

2.6.2 附加功能块

RTC - 本 PLC 不支持其功能。 INTEGRAL - 积分——积分功能块对输入 XIN 的值按时间进行积分。

```
( BOOL:RUN, BOOL:R1, REAL:XIN, REAL:X0, TIME:CYCLE ) => ( BOOL:Q, REAL:XOUT )
```

当输入 RUN 为 True 且强制输入 R1 为 False 时，XOUT 将根据采样周期 CYCLE 时间值随 XIN 变化。当 RUN 为 False 且强制输入 R1 为 True 时，XOUT 将保持最后的输出值。若 R1 为 True，则 XOUT 将被设置为 X0 的值。

```
XOUT = XOUT + (XIN * CYCLE)
```

DERIVATIVE - 微分

```
( BOOL:RUN, REAL:XIN, TIME:CYCLE ) => ( REAL:XOUT )
```

当 RUN 为 True 时，XOUT 将按 XIN 值的变化率、并依据采样周期 CYCLE 时间值成比例变化。

```
XOUT = ((3 * (XIN - XIN(to-3))) + XIN(to-1) - XIN(to-2)) / (10 * CYCLE)
```

PID - PID——PID 功能块为闭环控制提供经典的三项控制器。它不包含任何输出限制参数（死区、最小值、最大值等），也不包含实际过程控制中通常使用的其他参数（另见 PID_A）。

```
( BOOL:AUTO, REAL:PV, REAL:SP, REAL:X0, REAL:KP, REAL:TR, REAL:TD, TIME:CYCLE ) => ( REAL:XOUT )
```

当 AUTO 为 False 时，PID 功能块的 XOUT 将跟随 X0 的值。当 AUTO 为 True 时，XOUT 将根据误差值（PV 过程变量 - SP 设定值）、KP 比例常数、TR 复位时间、TD 微分常数以及采样周期 CYCLE 时间值进行计算。

```
XOUT = KP * ((PV-SP) + (I_OUT/TR) + (D_OUT * TD))
```

RAMP - 斜坡——以标准中给出的示例为模型，并增加了“保持回退”（Holdback）功能。

```
( BOOL:RUN, REAL:X0, REAL:X1, TIME:TR, TIME:CYCLE, BOOL:HOLDBACK, REAL:ERROR, REAL:PV ) => ( BOOL:RAMP, REAL:XOUT )
```

当 RUN 与 HOLDBACK 均为 False 时，XOUT 将跟随 X0 的值。当 RUN 为 True 且 HOLDBACK 为 False 时，XOUT 将在每个采样周期 CYCLE 时间值按 $OUT(to-1) + (X1 - XOUT(to-1))$ 变化。

HYSTERESIS - 滞环

```
( REAL:XIN1, REAL:XIN2, REAL:EPS ) => ( BOOL:Q )
```

当 XIN1 的值大于 XIN2 + EPS 的值时，Q 变为 True。当 XIN1 的值小于 XIN2 - EPS 的值时，Q 变为 False。

2.6.3 类型转换

TYPE[A]_TO_TYPE[B] - 数据类型转换

```
( TYPE[A]:IN ) => ( TYPE[B]:OUT )
```

```
ST 语法示例: OUT := TYPE[A]_TO_TYPE[B](IN1);
```

2.6.4 数值运算

ABS 绝对值；SQRT 平方根；LN 自然对数；LOG 以 10 为底的对数；EXP 指数；SIN 正弦；COS 余弦；TAN 正切；ASIN 反正弦；ACOS 反余弦；ATAN 反正切 各函数均具有相应签名与 ST 示例，例如：OUT := ABS(IN1); 等。

ABS - 绝对值

```
( ANY_NUM:IN ) => ( ANY_NUM:OUT )
```

```
ST 语法示例：OUT := ABS(IN1);
```

SQRT - 平方根

```
( ANY_REAL:IN ) => ( ANY_REAL:OUT )
```

```
ST 语法示例：OUT := SQRT(IN1);
```

LN - 自然对数

```
( ANY_REAL:IN ) => ( ANY_REAL:OUT )
```

```
ST 语法示例：OUT := LN(IN1);
```

LOG - 以 10 为底的对数

```
( ANY_REAL:IN ) => ( ANY_REAL:OUT )
```

```
ST 语法示例：OUT := LOG(IN1);
```

EXP - 指数

```
( ANY_REAL:IN ) => ( ANY_REAL:OUT )
```

```
ST 语法示例：OUT := EXP(IN1);
```

SIN - 正弦

```
( ANY_REAL:IN ) => ( ANY_REAL:OUT )
```

```
ST 语法示例：OUT := SIN(IN1);
```

COS - 余弦

```
( ANY_REAL:IN ) => ( ANY_REAL:OUT )
```

```
ST 语法示例：OUT := COS(IN1);
```

TAN - 正切

```
( ANY_REAL:IN ) => ( ANY_REAL:OUT )
```

```
ST 语法示例：OUT := TAN(IN1);
```

ASIN - 反正弦

```
( ANY_REAL:IN ) => ( ANY_REAL:OUT )
```

```
ST 语法示例：OUT := ASIN(IN1);
```

ACOS - 反余弦

```
( ANY_REAL:IN ) => ( ANY_REAL:OUT )
```

```
ST 语法示例：OUT := ACOS(IN1);
```

ATAN - 反正切

```
( ANY_REAL:IN ) => ( ANY_REAL:OUT )
```

```
ST 语法示例：OUT := ATAN(IN1);
```

2.6.5 算术运算

ADD 加法 - $OUT = IN1 + IN2$ 。输入数量可扩展。MUL 乘法 - $OUT = IN1 * IN2$ 。输入数量可扩展。SUB 减法 - $OUT = IN1 - IN2$ 。DIV 除法 - $OUT = IN1 / IN2$ 。例如 $1234 / 10 = 3$ 。MOD 取余（模运算） - $OUT = IN1 \text{ modulo } IN2$ 。例如 $1234 \text{ modulo } 10 = 4$ 。EXPT 幂运算 - $OUT = IN1 ** IN2$ 。例如 $2^3 = 8$ 。MOVE 赋值 - $OUT = IN$ 。

2.6.6 时间运算

ADD / ADD_TIME - 时间加法 (TIME, TIME) => TIME

ADD / ADD_TOD_TIME - 时刻加法 (TOD, TIME) => TOD

ADD / ADD_DT_TIME - 日期加法 (DT, TIME) => DT

MUL / MULTIME - 时间乘法 (TIME, ANY_NUM) => TIME

SUB / SUB_TIME - 时间减法

SUB / SUB_DATE_DATE - 日期减法 (DATE, DATE) => TIME

SUB / SUB_TOD_TIME - 时刻减法 (TOD, TIME) => TOD

SUB / SUB_TOD_TOD - 时刻减法 (TOD, TOD) => TIME

SUB / SUB_DT_TIME - 日期时间减法 (DT, TIME) => DT 以及 (DT, DT) => TIME

DIV / DIVTIME - 时间除法 (TIME, ANY_NUM) => TIME

2.6.7 位移运算

SHL 左移 - OUT 表示将 IN 变量左移 N 位。右侧以零填充。SHR 右移 - 左侧以零填充。ROR 循环右移 - 每次循环时，最右侧位填入最左侧位。ROL 循环左移 - 每次循环时，最左侧位填入最右侧位。

2.6.8 按位运算

AND 按位与；OR 按位或；XOR 按位异或；NOT 按位取反。在适用情况下，输入数量可扩展。

2.6.9 选择

SEL 二进制选择（二选一） - 若 G 为 False，OUT 跟随 IN0；若为 True，则跟随 IN1。MAX 最大值；MIN 最小值 - 输入数量可扩展。LIMIT 限幅 - OUT 在 MN 与 MX 之间跟随 IN。MUX 多路选择器（N 选 1） - 根据 K 的值，选择其中一个输入。

2.6.10 比较

GT 大于；GE 大于或等于；EQ 等于；LT 小于；LE 小于或等于；NE 不等于。

2.6.11 字符串

LEN 字符串长度 - 例如 'ABCDEFGH' => 8

LEFT 取字符串左侧 - 例如 L=3 => 'ABC'

RIGHT 取字符串右侧 - 例如 L=3 => 'FGH'

MID 取字符串中间部分 - L=4 P=2 => OUT 字符串将为 'DE'。

CONCAT 连接

CONCAT_DAT_TOD 时间连接 (DATE, TOD) => DT

INSERT 插入 DELETE 删除 REPLACE 替换

FIND 查找位置 - 例如在 'ABCDEFGF' 中查找 'DEF' => 4

2.6.12 本地 POU

LOGGER - 日志记录器 (BOOL:TRIG, STRING:MSG, LOGLEVEL:LEVEL) => ()

Logger 用于离线日志记录。由触发 (trig)、消息 (message) 和日志级别 (log level) 定义的日志数据保存在 MCU 数据库中。日志数据可从 PLC Log (2.10 PLC Log) 读取。Level 必须是介于 0 与 3 之间的数字，并以 Expression 形式写入。

2.6.13 LPC POU

DEW_POINT - 根据温度和湿度计算露点。GET_RETAIN_DATA - 当 READ 为接通时，从保持数据库中固定位置 INDEX (0-1999) 获取三个变量。主要用于 RFID 钥匙卡数据。SET_RETAIN_DATA - 当 WRITE 为接通时，将三个变量写入保持数据库。FIND_RETAIN_DATA - 在 START_IDX 与 END_IDX 之间搜索三个变量是否存在于保持数据库中。PID_A - PID A 版本包含自动化 PID 过程控制中最常用的参数。参数说明如下：SET_POINT——设定值；ACTUAL_VALUE——实际值；REVERSE——反向作用；HOLD——保持；MANUAL——手动；MANUAL_REF——手动参考值；Kp_GAIN——比例增益；Ki_GAIN——积分增益；Kd_GAIN——微分增益；Td_FADEOUT——微分衰减；DEAD_BAND——死区；OUT_MAX——输出上限；OUT_MIN——输出下限；CYCLE_TIME——循环时间。

2.6.14 用户自定义 POU

所有用户自定义函数与功能块均添加至该库组中。

2.7 Debugger (调试器)

Debugger (调试器) 可以以数值和图形方式显示所选变量的实际在线值。可通过在 Project (项目) 选项卡 (instances 窗口) 中点击变量的眼镜图标，或在编辑器工作区中双击变量，将要显示的变量添加到此处。变量的实际值可呈现为：

- 数值显示，
- 一维曲线图，可包含一个或多个变量（将变量拖放到现有曲线图的右侧），以及
- 多维曲线图（将变量拖放到现有曲线图的左侧）。

数值趋势在可选时间范围内显示 (10ms、...、1s、...、1m、...、1h、...、24h)。该时间范围对所有被监视的变量通用。变量也可导出到剪贴板。在此窗口中还可通过点击 padlock (锁) 图标对这些变量进行强制 (或释放强制) (将鼠标指针移到数值上并点击锁定的 padlock 图标)。

2.8 Search (搜索)

Search (搜索) —— Search 窗口显示来自 Edit (编辑) 工具栏中 Search in Project (在项目中搜索) 请求的结果。可选择自定义搜索模式和不同的搜索范围。

2.9 Console (控制台)

Console (控制台) —— 包含 LPC Manager 在处理系统程序活动 (build、transfer、debugging、communication 等) 期间的活动日志。

2.10 PLC Log (PLC 日志)

PLC Log (PLC 日志) —— 包含由 logger (日志) 功能记录并存储在 MCU 上的日志数据。日志数据按级别类型过滤。级别类型由标志指示 (error 标志 = 级别 0, warning 标志 = 级别 1, information 标志 = 级别 2, status 标志 = 级别 3)。

3 编程语言

LPC Manager 基于 PLC（可编程逻辑控制器）编程语言国际标准 IEC 61131-3。可使用以下类型的 PLC 编程语言：文本类：

- IL Instruction List（指令表）
- ST Structured Text（结构化文本）

图形类：

- FBD Function Block Diagram（功能块图）
- LD Ladder Diagram（梯形图）
- SFC Sequential Function Chart（顺序功能图）

3.1 IL - Instruction List（指令表）

Instruction List（指令表）编程语言由一系列指令组成。它类似于汇编语言。每条指令应从新行开始，并包含一个带有可选修饰符的操作符；若该操作需要，还可包含一个或多个用逗号分隔的操作数。操作数可以是字面量、枚举值和变量。

3.2 ST - Structured Text（结构化文本）

Structured Text（结构化文本）编程语言由一系列指令组成。它是一种类似 C 的高级语言。文本行的结束应被视为与空格（SP）字符相同。

（注：原文包含一页 ST 语法示例截图——ST 语法示例请参见英文原版中的截图。）

3.3 FBD - Function Block Diagram（功能块图）

Function Block Diagram（功能块图）编程语言是一种用于可编程控制器编程的图形语言。FBD 语言的元件应通过信号流线相互连接。功能块的输出不得相互连接。特别是，LD 语言中的“wired-OR”（线或）结构在 FBD 语言中不允许使用。应改用显式的布尔“OR”功能块。

3.4 LD - Ladder Diagram（梯形图）

Ladder Diagram（梯形图）编程语言是一种用于可编程控制器编程的图形语言。它使可编程控制器能够通过标准化图形符号对数据进行测试和修改。这些符号以网络形式布置，类似于继电器梯形逻辑图中的一个“梯级（rung）”。LD 网络的左右两侧由电源轨（power rails）界定。

3.5 SFC - Sequential Function Chart（顺序功能图）

Sequential Function Chart（顺序功能图）编程语言是一种用于可编程控制器编程的图形语言。顺序功能图元素用于对本文件所述的一种 PLC 编程语言所编写的可编程控制器程序组织单元（POU）进行内部组织结构划分，以实现顺序控制功能。SFC 元素提供了一种将可编程控制器程序组织单元划分为一组步骤（steps）和转换（transitions）的方法，这些步骤与转换通过有向连接相互连接。每个步骤关联一组动作（actions），每个转换关联一个转换条件。由于 SFC 元素需要存储状态信息，因此可使用这些元素进行结构化的程序组织单元仅限于功能块（function blocks）和程序（programs）。如果程序组织单元的任何部分被划分为 SFC 元素，则整个程序组织单元都必须如此划分。如果未对程序组织单元给出 SFC

划分，则整个程序组织单元应被视为在调用实体控制下执行的单个动作。

附录 A – 错误报告

如果您认为在我们的软件中发现了缺陷（bug），或有改进或新增功能的想法，欢迎与我们分享您的意见（support@smarteht.com）。我们将评估可行性，并尽量在后续版本中纳入。

您应联系供应商并提供说明。应包含以下信息：

- 软件版本。
- 缺陷或想法的详细描述。
- 如有可能，能够复现问题的步骤（若报告的是缺陷）。
- 您的联系信息（电子邮件、电话、传真）。

如需更多信息，我们可能需要在确定确切解决方案之前与您联系。请记住：唯一不需要维护的软件，是未被使用的软件！

附录 B – 文档历史

下表描述了本文档的所有变更。

日期	版本	说明
----	----	----

30.09.2011	-	初步版本，以 LPC Manager User Manual 形式发布。
------------	---	--------------------------------------

30.01.2012	001	首次正式发布。
------------	-----	---------

30.06.2012	002	相对上一版本的变更。
------------	-----	------------

25.05.2014	003	根据 LPC Composer 5.0.1.32 新版本进行更新。
------------	-----	-----------------------------------

22.01.2016	004	根据 LPC Smarteh IDE 5.1.4.2 新版本进行更新。
------------	-----	-------------------------------------

译者说明

本中文版为 OpenPLC Editor 仓库收录的英文用户手册（原 LPC Manager User Manual，基于 Beremiz）的简体中文翻译。界面截图仍请参考英文原版 PDF。技术标识符、标准编号与函数名保持英文原文。